

Lecture 08

Singular Value Decomposition, PCA, Low-Rank Approximation, and Data Compression

Instructor: Dr. M. Asad Iqbal, Assistant Professor

Department of Information Technology

Semester: 4th Semester, BS IT

Lecture Focus

Decomposing any data matrix into orthogonal directions, identifying informative components, and building compact approximations for dimensionality reduction and compression. [\[Data Eng. Link: Data Compression, Dimensionality Reduction\]](#)

1 Lecture Overview

Eigenvalue decomposition is powerful but applies directly only to square matrices and may not provide an orthogonal basis. Singular value decomposition works for every real or complex matrix and exposes its most important input and output directions.

SVD supports least squares, pseudoinverses, matrix rank, PCA, recommendation systems, latent semantic analysis, denoising, and image compression. It is one of the most useful tools in applied linear algebra and machine learning.

Learning Outcomes

After completing this lecture, students should be able to:

- State the full and compact forms of the SVD.
- Interpret left and right singular vectors geometrically.
- Connect singular values with eigenvalues of $A^T A$ and AA^T .
- Determine rank, condition number, and pseudoinverse from the SVD.
- Construct a truncated SVD and low-rank approximation.
- Explain the Eckart-Young optimality result.
- Derive PCA from the SVD of centered data.
- Calculate explained-variance ratios.
- Implement PCA and image compression using Python.

Why SVD Is So General

Property	SVD advantage
Matrix shape	Works for square and rectangular matrices
Existence	Every matrix has an SVD
Geometry	Uses orthonormal input and output directions
Rank	Number of positive singular values
Compression	Orders components by decreasing strength
Numerical analysis	Reveals ill-conditioning and near dependence

2 The Singular Value Decomposition

For $A \in \mathbb{R}^{m \times n}$, the full SVD is

$$A = U \Sigma V^T,$$

where

$$U \in \mathbb{R}^{m \times m}, \quad \Sigma \in \mathbb{R}^{m \times n}, \quad V \in \mathbb{R}^{n \times n}.$$

The columns of U and V are orthonormal:

$$U^T U = I, \quad V^T V = I.$$

The diagonal entries of Σ are the singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

where $r = \text{rank}(A)$.

2.1 Compact SVD

Keeping only the r positive singular values gives

$$A = U_r \Sigma_r V_r^T,$$

where

$$U_r \in \mathbb{R}^{m \times r}, \quad \Sigma_r \in \mathbb{R}^{r \times r}, \quad V_r \in \mathbb{R}^{n \times r}.$$

2.2 Outer-Product Form

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

Each term is a rank-one matrix. The singular value σ_i measures the strength of that component.

Three-Stage Transformation

For $A\mathbf{x} = U\Sigma V^T\mathbf{x}$:

1. V^T expresses the input in right-singular-vector coordinates.
2. Σ scales those coordinates by the singular values.
3. U places the result in the output space.

3 Connection with Eigenvalues and Subspaces

From $A = U\Sigma V^T$:

$$A^T A = V\Sigma^T \Sigma V^T, \quad AA^T = U\Sigma \Sigma^T U^T.$$

Therefore,

$$A^T A \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i, \quad AA^T \mathbf{u}_i = \sigma_i^2 \mathbf{u}_i.$$

The right singular vectors are eigenvectors of $A^T A$, the left singular vectors are eigenvectors of AA^T , and singular values are square roots of the non-zero eigenvalues.

3.1 Fundamental Subspaces

- $\mathbf{v}_1, \dots, \mathbf{v}_r$ form an orthonormal basis for $\text{Row}(A)$.
- $\mathbf{u}_1, \dots, \mathbf{u}_r$ form an orthonormal basis for $\text{Col}(A)$.
- Remaining right singular vectors span $\text{Null}(A)$.
- Remaining left singular vectors span $\text{Null}(A)$.

3.2 Condition Number

For a full-rank matrix,

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}}.$$

A large ratio indicates that some directions are transformed much more weakly than others, making inversion sensitive to noise.

3.3 Moore-Penrose Pseudoinverse

$$A^+ = V\Sigma^+ U^T,$$

where each positive singular value is replaced by its reciprocal. The pseudoinverse extends inverse-like computation to rectangular and singular matrices.

[ML/DS Connection: The pseudoinverse returns a minimum-norm least-squares solution. Small singular values reveal unstable directions and motivate truncation or regularization.]

4 Worked Rank-One Example

Consider

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Both columns are identical, so $\text{rank}(A) = 1$.

One compact SVD is

$$U = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}, \quad V = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Therefore,

$$A = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

The second singular value is zero. The direction

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

lies in the null space because $A\mathbf{v}_2 = \mathbf{0}$.

4.1 Geometric Interpretation

The direction $[1, 1]^T$ is scaled by 2. The perpendicular direction $[1, -1]^T$ is completely removed. Every output lies on the line spanned by $[1, 1]^T$.

5 Low-Rank Approximation

The rank- k truncated SVD is

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T = U_k \Sigma_k V_k^T, \quad k < r.$$

It keeps the strongest components and discards weaker directions.

5.1 Eckart-Young Theorem

Among all matrices B of rank at most k , A_k is a best approximation to A under both the spectral and Frobenius norms:

$$\begin{aligned} \|A - A_k\|_2 &= \sigma_{k+1}, \\ \|A - A_k\|_F &= \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}. \end{aligned}$$

5.2 Retained Energy

A common component-selection measure is

$$\text{retained energy}(k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}.$$

5.3 Storage Comparison

An $m \times n$ matrix stores mn values. A rank- k SVD representation stores approximately

$$k(m + n + 1)$$

values: U_k , the k singular values, and V_k .

Compression is beneficial when $k(m + n + 1) \ll mn$.

[ML/DS Connection: Low-rank models capture latent structure in images, user-item matrices, text-term matrices, and embeddings. The approximation replaces many original variables with a smaller set of learned factors.]

6 Principal Component Analysis through SVD

Let $X \in \mathbb{R}^{m \times n}$ contain observations as rows and features as columns. First center every feature:

$$X_c = X - \mathbf{1}\mu^T.$$

Compute

$$X_c = U\Sigma V^T.$$

6.1 PCA Quantities

- Principal directions: columns of V .
- Component scores: $Z = X_c V = U \Sigma$.
- Sample covariance: $C = X_c^T X_c / (m - 1)$.
- Explained variance: $\lambda_i = \sigma_i^2 / (m - 1)$.
- Explained-variance ratio:

$$\frac{\sigma_i^2}{\sum_j \sigma_j^2}.$$

Keeping the first k components gives

$$Z_k = X_c V_k$$

and the reconstruction

$$\hat{X} = Z_k V_k^T + \mathbf{1} \mu^T.$$

6.2 Centering and Scaling

- Centering is essential because PCA studies variation around the mean.
- Standardization is often appropriate when features use different units or scales.
- Standardization is not automatic; it changes the question from covariance structure to correlation-like structure.

[ML/DS Connection: PCA is unsupervised: it preserves directions of large input variance, not necessarily directions most useful for predicting a target. PCA supports visualization, compression, noise reduction, preprocessing, and multicollinearity management.]

7 Python and NumPy Practice

7.1 Computing and Verifying an SVD

```

1 import numpy as np
2 A = np.array([[3.0, 1.0, 1.0], [-1.0, 3.0, 1.0]])
3 U, singular_values, Vt = np.linalg.svd(A, full_matrices=False)
4 A_reconstructed = (U * singular_values) @ Vt
5 print("Singular values:", singular_values)
6 print("Reconstruction correct:", np.allclose(A, A_reconstructed))
7 print("Rank:", np.linalg.matrix_rank(A))
8 print("Condition number:", np.linalg.cond(A))

```

7.2 Truncated SVD Function

```

1 def truncated_svd(A, k):
2     U, s, Vt = np.linalg.svd(A, full_matrices=False)
3     if not (1 <= k <= len(s)):
4         raise ValueError("k is outside the valid range.")
5     return (U[:, :k] * s[:k]) @ Vt[:k, :]
6
7 A_rank1 = truncated_svd(A, k=1)
8 relative_error = (
9     np.linalg.norm(A - A_rank1, "fro") / np.linalg.norm(A, "fro")
10 )
11 print("Rank-1 approximation:\n", A_rank1)
12 print("Relative error:", relative_error)

```

7.3 Pseudoinverse Check

```

1 A_plus = np.linalg.pinv(A)
2 print(np.allclose(A @ A_plus @ A, A))

```

8 Applications: PCA and Image Compression

8.1 PCA with NumPy

```

1 X = np.array([
2     [2.0, 0.0],
3     [3.0, 1.0],
4     [4.0, 1.0],
5     [5.0, 3.0],
6     [6.0, 4.0]
7 ])
8 mean = X.mean(axis=0)
9 X_centered = X - mean
10 U, s, Vt = np.linalg.svd(X_centered, full_matrices=False)
11 explained_ratio = s**2 / np.sum(s**2)
12 scores_1d = X_centered @ Vt[:, 0]
13 X_reconstructed = scores_1d[:, None] @ Vt[:, 0:1] + mean
14 print("Explained ratio:", explained_ratio)
15 print("Reduced shape:", scores_1d.shape)
16 print("Reconstructed data:\n", X_reconstructed)

```

8.2 Compressing a Handwritten-Digit Image

```

1 from sklearn.datasets import load_digits
2 digits = load_digits()
3 image = digits.images[0]
4 U, s, Vt = np.linalg.svd(image, full_matrices=False)
5 k = 2
6 compressed = (U[:, :k] * s[:k]) @ Vt[:k, :]

```

```
7 original_values = image.size
8 svd_values = k * (image.shape[0] + image.shape[1] + 1)
9 relative_error = (
10     np.linalg.norm(image - compressed, "fro") /
11     np.linalg.norm(image, "fro")
12 )
13 print("Original storage:", original_values)
14 print("SVD storage:", svd_values)
15 print("Relative error:", relative_error)
```

[ML/DS Connection: The compressed image keeps the strongest spatial patterns while discarding weaker components. Larger k improves visual quality but reduces the storage advantage.]

9 Review and Independent Practice

Practice Tasks

1. State the dimensions of U , Σ , and V for the full SVD of a 5×3 matrix.
2. Explain how singular vectors relate to the four fundamental subspaces.
3. Find a compact SVD of $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ and verify its rank-one reconstruction.
4. Use singular values to calculate rank and condition number for a matrix of your choice.
5. Construct rank-1 and rank-2 approximations of a matrix and compare their Frobenius errors.
6. Apply PCA to a dataset with at least three features. Report the number of components needed for 90% explained variance.
7. Compress a grayscale image for several values of k and compare storage with reconstruction error.

Key Takeaways

- Every matrix has an SVD with orthonormal input and output directions.
- Singular values measure the strength of independent matrix components.
- SVD reveals rank, subspaces, conditioning, and pseudoinverse structure.
- Truncated SVD gives an optimal low-rank approximation.
- PCA is obtained from the SVD of a centered data matrix.
- Explained variance helps select components but must be balanced with task performance.
- Compression replaces the original matrix with a smaller factor representation.

Key Terms

Singular value, Left singular vector, Right singular vector, Compact SVD, Pseudoinverse, Condition number, Truncated SVD, Low-rank approximation, Frobenius norm, PCA, Explained variance, Reconstruction error.